NorthRiver Data Warehouse: Summary Report

Star schema, automated Databricks pipeline and Power BI revenue analytics

1. Problem framing

NorthRiver needed to replace fragmented transactional reporting with one trusted source for revenue analytics. The warehouse has to consolidate orders, products, customers and sales channels, answer revenue questions by product category, region, channel and time, and keep history accurate when customer and product attributes change. It also has to avoid common reporting errors: silently dropped rows in joins, revenue inflated by duplicate keys, and rates such as unit price being added together instead of averaged.

2. Architecture summary

Data starts in the NorthRiver PostgreSQL OLTP database (customers, products, channels, orders and order lines). Extract notebooks copy it into staging tables in Databricks, where it is cleaned and typed. Transform and load notebooks in Databricks then build the star schema in the **northriver_warehouse** schema, assigning surrogate keys and applying SCD Type 2 logic. A Databricks job, **northriver_nightly_pipeline**, runs the extract, transform and load tasks in order every night at 02:00 (Europe/Berlin). Power BI connects to the warehouse and serves the revenue dashboard.

3. Main design choices

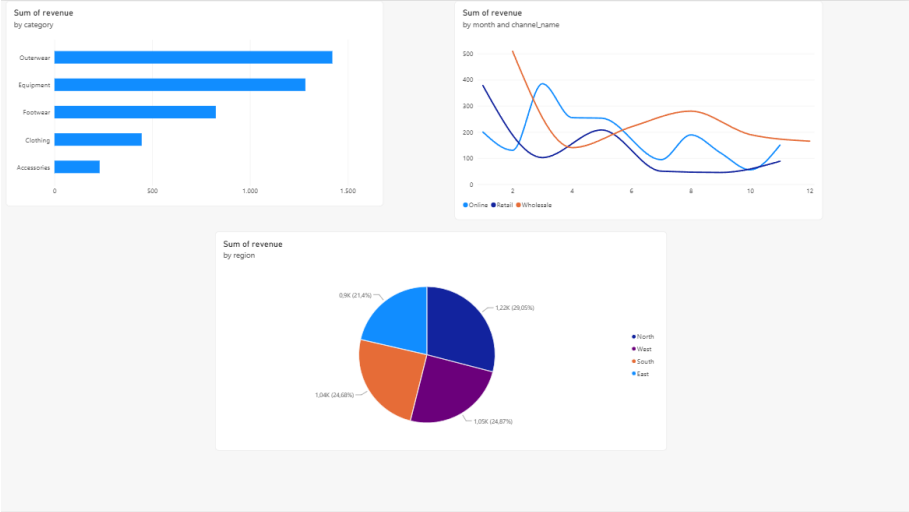
Decision	Choice and rationale
Fact grain	**fact_orders**: one row per order line item (one product on one order). The atomic grain supports any rollup by date, product, channel or customer without losing detail. Revenue is net_amount (extended amount less discount).
Dimensions	Four dimensions around the fact: **dim_customer** (segment, region), **dim_product** (category and cost attributes), **dim_channel** and **dim_date**. Integer surrogate keys (_sk) separate the warehouse from source-system keys.
SCD Type 2	Applied to **dim_customer** and **dim_product** using effective_from, effective_to and is_current. Each order links to the version that was valid when it was placed, so history is not rewritten. Channel and date are stable lookups and need no history.
Loading order	Dimensions load first so surrogate keys exist; **fact_orders** loads last and looks up the correct keys. The Databricks job enforces this order.

4. Validation evidence

The following checks were run on the loaded warehouse and all passed:

- **Grain and reconciliation:** the row count of fact_orders matches the source line-item count, and counting distinct order_id isolates unique orders.
- **Join completeness:** comparing total rows with rows matched to a dimension key showed no silent row drops. Late-arriving orders are kept under NULL dimension values rather than discarded.
- **Fan-out prevention:** surrogate-key joins are one-to-one or one-to-many, so revenue totals are not inflated.
- **Aggregate integrity:** total revenue was reconciled against the source, and unit price is averaged, never summed.
- **SCD Type 2 behaviour:** after changing a customer attribute in the source and rerunning the pipeline, the old version was closed and a new current version was inserted.
- **Automation:** recent runs of northriver_nightly_pipeline completed successfully, with all three tasks (extract, transform, load) passing.

The Power BI dashboard built on the warehouse shows revenue by product category, by region, and by month and channel, which confirms the model answers the business questions it was designed for.



Power BI dashboard: revenue by category, by month and channel, and by region.

5. Limitation and next improvement

Dimensions are synchronised in batches, so when orders arrive faster than dimension updates, a few lines can briefly lack a matching dimension row. The next improvement is late-arriving dimension handling with a fallback surrogate key (for example -1 for unknown or in-flight), plus automated schema-drift detection in the staging layer and customer cohort retention metrics in Power BI.